Module-wise Practical Implementation Notes

Module: Natural Language Processing (NLP)

Started coding practice with basic NLP concepts. In the lectures, we learned about text processing before feeding it to AI. I wrote some Python code using libraries like NLTK to practice Tokenization and Stop-word removal.

Understanding TF-IDF & Word2Vec

Practiced the mathematical intuition behind TF-IDF (Term Frequency - Inverse Document Frequency). I wrote a small Python script using scikit-learn to convert a list of sentences into a matrix of TF-IDF features. It was amazing to see how words get numerical weights based on their rarity across documents.

For Word2Vec, I tested how words are mapped into vector spaces. I didn't train a full model locally because it requires heavy compute, but I loaded a pre-trained gensim model to test word similarities (like finding out that 'King' - 'Man' + 'Woman' = 'Queen' in vector math). This cleared my foundation before jumping into LLMs.

Module: LLM Basics & Google PaLM API

Transitioned from traditional NLP to Generative AI. I created an account on Google Cloud / Google AI Studio to get my API keys. My first practice was with the Google PaLM API.

```
import google.generativeai as palm
palm.configure(api_key="MY_SECRET_KEY") response =
palm.chat(messages="Hello, I am testing the API")
print(response.last)
```

Initially, I faced an authentication error because I hardcoded the API key. I learned the best practice is to use python-dotenv and store the key in a .env file. Fixed the error and successfully got a response in the terminal.

Module: Exploring Gemini API (Parts 1 to 3)

The lectures on Gemini were very detailed. I upgraded my practice scripts from PaLM to the newer Gemini API. I tested both text-only prompts (gemini-pro) and multimodal prompts.

I wrote a practice script to test system instructions. I gave Gemini the role of a sarcastic customer support agent just for fun during practice, and it worked perfectly. Then I switched it to a polite, professional assistant which we actually need for the final project. I learned how to pass 'history' in the chat session so the model remembers the previous questions.

Module: Creation of Chatbot (Parts 1 to 3)

This is where everything came together. I started building the actual user interface for the bot. Instead of writing raw HTML/CSS from scratch which takes a lot of time, I practiced using Streamlit in Python.

```
import streamlit as st st.title("Customer Support Bot") user_input
= st.chat_input("Ask your question here...") if user_input:
st.write("User: ", user_input) # Passed to Gemini API here
```

I faced a bug where the Streamlit page would refresh and delete the chat history every time I hit enter. To fix this, I learned about Streamlit's Session State (st.session_state). I created a list to store all messages, so even when the page reloads, the chat history stays on the screen.

Module: Finalizing the Customer Service Bot

In the final practice sessions, I integrated the Gemini Chat Object with the Streamlit UI. I wrote a strict system prompt limiting the bot to only answer queries related to our business domain. If I asked it about sports or weather, the bot politely declined, saying it can only help with customer service issues. This was exactly what was taught in the final modules.

The integration was a success. The practice codes taught me how text goes from the front-end (HTML/Streamlit) -> to the Python backend -> to the Gemini API -> and back to the user seamlessly.

All modules practiced locally. Ready for final deployment.